

Model Question Paper

APJ ABDUL KALAM TECHNOLOGICAL UNIVERSITY
SIXTH SEMESTER B.TECH DEGREE EXAMINATION

Course Code: CST 302
Course Name: Compiler Design

PART A
Answer All Questions. Each Question Carries 3 Marks

1. Specify the analysis and synthesis parts of compilation.

Compilation is the process of translating high-level programming languages into machine code that can be executed by a computer. The compilation process can be divided into two main stages: analysis and synthesis.

Analysis is the first stage of the compilation process, during which the compiler analyzes the source code to ensure that it is syntactically and semantically correct. The analysis phase includes the following steps: Lexical Analysis, Syntax Analysis, Semantic Analysis and Intermediate Code Generation.

Synthesis is the second stage of the compilation process, during which the compiler generates the machine code that can be executed by a computer. The synthesis phase includes the following steps: Code Optimization, Code Generation and Symbol Table Management.

Overall, the analysis and synthesis phases work together to transform high-level source code into low-level machine code that can be executed by a computer.

2. Define the terms token, lexemes and patterns with examples.

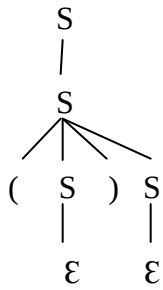
Token: Token is a sequence of characters that can be treated as a single logical entity. Typical tokens are, Identifiers, keywords, operators, special symbols, number etc. For example, in the statement "x = 5 + y;" the tokens are "identifier", "assign_operator", "number", "operator", "identifier", and "punctuation" respectively.

Pattern: A set of strings in the input for which the same token is produced as output. This set of strings is described by a rule called a pattern associated with the token. For example, the pattern for a valid integer token is "one or more digits".

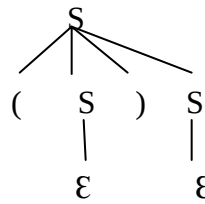
Lexeme: A lexeme is a sequence of characters in the source program that is matched by the pattern for a token. For example, in the statement "x = 5 + y;" the lexemes are "x", "=", "5", "+", "y", and ";".

3. Is the grammar $S \rightarrow S \mid (S)S \mid \epsilon$ ambiguous? Justify your answer.

Consider the string "()" which can be generated in two different ways:



Parse Tree 1



Parse Tree 2

Therefore, the grammar is ambiguous since the string "()" can be derived by more than one parse tree.

4. What is left recursive grammar? Give an example. What are the steps in removing left recursion?

A context free grammar is said to be left recursive if it has a non terminal A such that there is a derivation $A \Rightarrow A\alpha$ for some string α . Here Leftmost symbol of the body is same as the non terminal at the head of the production.

Left recursion can cause problems in parsing and may make it difficult to generate correct and efficient parsers.

Here is an example of a left-recursive grammar:

$A \rightarrow A + B \mid B$

$B \rightarrow C * B \mid C$

$C \rightarrow (A) \mid id$

The non-terminal symbol A can derive a string that starts with itself, for example, $A \Rightarrow A + B \Rightarrow A + C * B \Rightarrow \dots$, creating an infinite loop.

To remove left recursion from a grammar, we can follow these steps:

Identify the left-recursive non-terminals.

For each left-recursive non-terminal A, create a new non-terminal A' and move all the productions of A that start with A to the new non-terminal A', replacing A with A' in the rest of the productions.

Thus the pair of production $A \rightarrow A\alpha \mid \beta$ could be replaced by non-left recursive productions

$$A \rightarrow \beta A'$$

$$A' \rightarrow \alpha A' | \epsilon$$

Without changing the strings derived from A.

5. Compare different bottom-up parsing techniques.

Different Bottom-Up Parsing Techniques are:

- a. Shift Reduce Parsing
- b. Operator Precedence Parsing
- c. LR Parsing
 - i. Simple LR Parser (SLR Parser)
 - ii. Canonical LR (CALR Parser)
 - iii. LALR Parser

Shift-reduce parsing is a general term for parsing techniques that shift input symbols onto a stack and then reduce them based on rules.

The main advantage of shift-reduce parsing is its flexibility, as it allows for incremental parsing and can handle a broad class of grammars.

However, the drawback is that shift-reduce parsers may suffer from conflicts (reduce-reduce and shift-reduce conflicts) when dealing with ambiguous grammars.

SLR parsing: SLR parsers have the least parsing power among the three techniques. They can handle a subset of the grammars that CLR and LALR parsers can handle.

CLR parsing: CLR parsers are the most powerful and can handle a broad range of grammars, including left-recursive and ambiguous grammars.

LALR parsing: LALR parsers have intermediate parsing power, more powerful than SLR but less powerful than CLR. They can handle a larger set of grammars compared to SLR parsers.

6. What are the possible actions of a shift reduce parser.

Shift: This action pushes the next input symbol onto the stack and moves to the next input symbol. This is done when the parser encounters a terminal symbol that is not part of a completed handle.

Reduce: This action applies a grammar production to the top symbols on the stack, replacing them with a non-terminal symbol that represents the completed handle. This is done when the

parser recognizes a sequence of symbols on the stack that matches the right-hand side of a production.

Accept: This action indicates that the input string has been successfully parsed and that the parsing process should terminate.

Error: This action indicates that the input string cannot be parsed according to the grammar, either because it contains a syntax error or because the grammar is ambiguous. The parser typically backtracks and tries different paths until it finds a valid parse or reports a syntax error.

7. Differentiate synthesized and inherited attributes with examples.

Synthesized attributes: Synthesized attributes are attributes that are defined at the bottom of the parse tree and are propagated up towards the root. Synthesized attributes are typically computed by applying grammar rules in a bottom-up manner, starting from the leaves and working towards the root.

Example: Consider the following grammar for arithmetic expressions:

$\text{Expr} \rightarrow \text{Expr} + \text{Term} \mid \text{Term}$

$\text{Term} \rightarrow \text{Term} * \text{Factor} \mid \text{Factor}$

$\text{Factor} \rightarrow (\text{Expr}) \mid \text{Number}$

The values of the synthesized attributes are computed as follows:

$\text{Factor.value} = \text{Number.value}$

$\text{Factor.value} = \text{Expr.value}$

$\text{Term.value} = \text{Factor.value} * \text{Term.value}$

$\text{Term.value} = \text{Factor.value}$

$\text{Expr.value} = \text{Term.value} + \text{Expr.value}$

$\text{Expr.value} = \text{Term.value}$

Inherited attributes: Inherited attributes are attributes that are defined at the top of the parse tree and are propagated down towards the leaves. They are used to provide information to the parse tree nodes from their parent nodes. Inherited attributes are typically computed by applying grammar rules in a top-down manner, starting from the root and working towards the leaves.

Example

- Using inherited attributes to parse a declaration and add type information to the symbol table.

Production	Semantic Rules
$D \rightarrow TL$	$L.in := T.type$
$T \rightarrow \text{int}$	$T.Type := \text{integer}$
$T \rightarrow \text{real}$	$T.Type := \text{real}$
$L \rightarrow L_1, id$	$L_1.in := L.in$ $addtype(id.entry, L.in)$
$L \rightarrow id$	$addtype(id.entry, L.in)$

8. Translate $a[i] = b * c - b * d$, to quadruple.

Assuming that a, b, c, and d are variables, the given expression $a[i] = b * c - b * d$ can be translated into quadruples as follows:

1. $t1 = b * c$ // Multiply b and c, store result in t1
2. $t2 = b * d$ // Multiply b and d, store result in t2
3. $t3 = t1 - t2$ // Subtract t2 from t1, store result in t3
4. $a[i] = t3$ // Store value of t3 in array element a[i]

Operator	Operand 1	Operand 2	Result
*	b	c	t1
*	b	d	t2
-	t1	t2	t3
=	t3		a[i]

9. What is the role of peephole optimization in the compilation process

Peephole optimization is a local optimization technique that is applied to the generated code by a compiler after the code generation phase. It works by analyzing a small sequence of instructions, typically a few adjacent instructions or a "peephole", and tries to improve the code by making small changes that don't affect the overall functionality of the program. The goal of peephole optimization is to generate more efficient code that executes faster or takes up less memory. Peephole optimization is a powerful optimization technique that can significantly improve the performance and efficiency of the generated code. However, it is a local optimization technique, and its effectiveness is limited to the code within the peephole. To achieve global optimization, compilers use other techniques such as loop optimization, function inlining, and inter procedural analysis.

10. What are the issues in the design of a code generator

The following issues arise during the code generation phase :

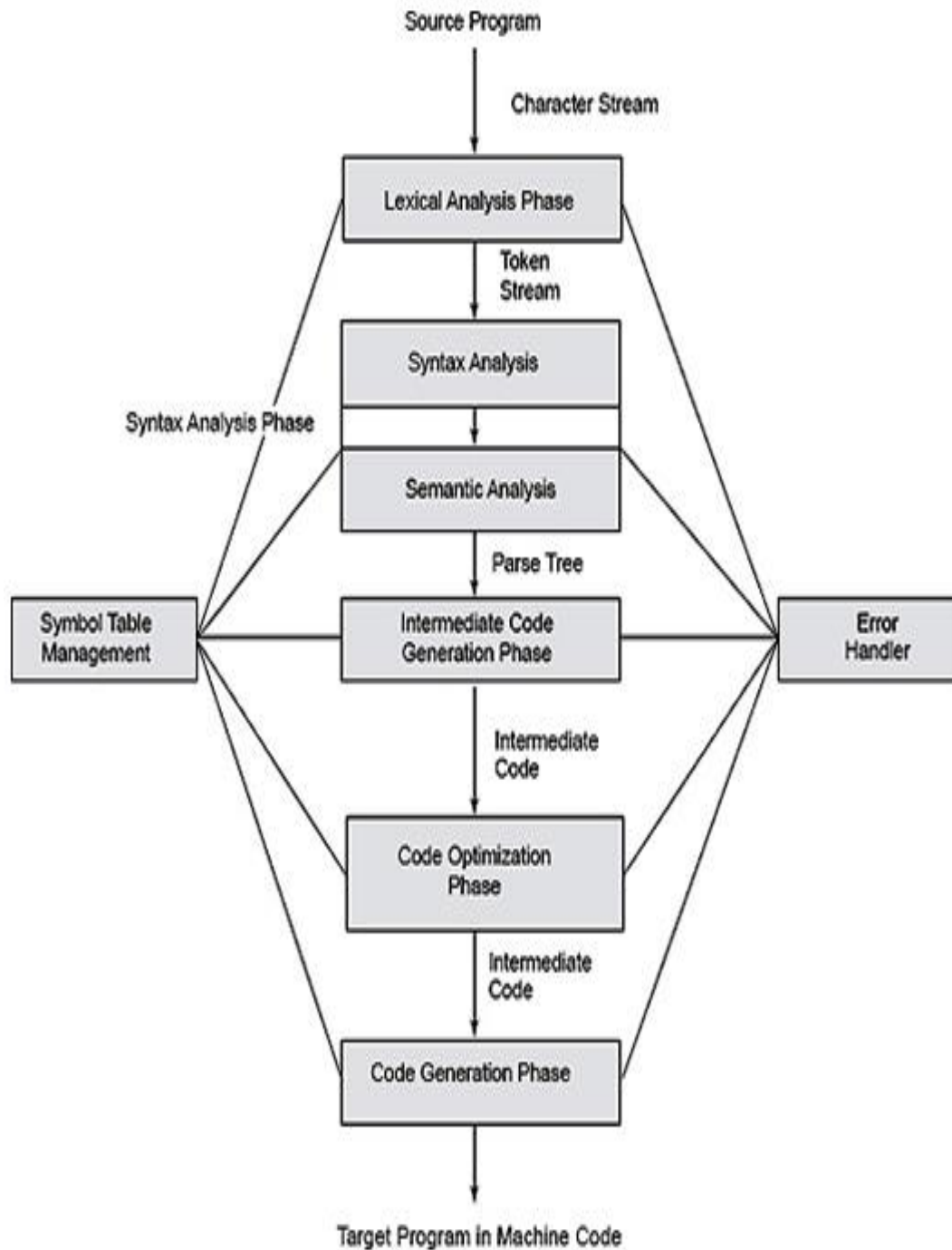
1. Input to code generator
2. Target program
3. Memory management
4. Instruction selection
5. Register allocation
6. Evaluation order

The code generator must take into account the target platform and architecture on which the generated code will run. Different platforms may have different instruction sets, memory models, and calling conventions, requiring specific considerations during code generation.

Part B

(Answer any one question from each module. Each question carries 14 Marks)

11. a Explain the different phases of a compiler with a running example. (9)



Lexical Analysis

- The first phase of a compiler is called *lexical analysis or scanning or linear analysis*.
- The lexical analyzer reads the stream of characters making up the source program and groups the characters into meaningful sequences called *lexemes*.
- For each lexeme, the lexical analyzer produces as output a token of the form

<token-name, attribute-value>

The first component token-name is an abstract symbol that is used during syntax analysis, and the second component attribute-value points to an entry in the symbol table for this token. Information from the symbol-table entry is needed for semantic analysis and code generation.

For example, if a source program contains the assignment statement
position = initial + rate * 60

Blanks separating the lexemes would be discarded by the lexical analyzer. The sequence of tokens produced as follows after lexical analysis.

<id, 1> <=> <id, 2> <+> <id, 3> <*> <60>

Here,

- id is an abstract symbol standing for identifier.
- 1,2 and 3 are points to the symbol table entries for position, initial and rate respectively.
- +,= and * are abstract symbols for addition, assignment and multiplication operators. They do not need any attribute-value.
- 60 is a lexeme that is mapped into the token <60>.

Syntax Analysis

- The second phase of the compiler is *syntax analysis or parsing or hierarchical analysis*.
- The parser uses the tokens produced by the lexical analyzer to create a tree-like intermediate representation that depicts the grammatical structure of the token stream.
- The hierarchical tree structure generated in this phase is called parse tree or syntax tree.

- In a syntax tree, each interior node represents an operation and the children of the node represent the arguments of the operation.

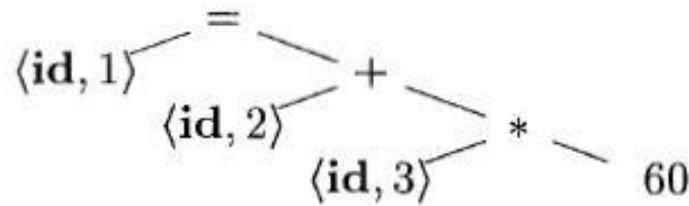


Figure: Syntax tree for position = initial + rate * 60

The tree has an interior node labeled * with <id, 3> as its left child and the integer 60 as its right child. The node <id, 3> represents the identifier rate. Similarly <id, 2> and <id, 1> are represented as in tree. The root of the tree, labeled =, indicates that we must store the result of this addition into the location for the identifier position.

Semantic Analysis

- The semantic analyzer uses the syntax tree and the information in the symbol table to check the source program for semantic consistency with the language definition.
- It also gathers type information and saves it in either the syntax tree or the symbol table, for subsequent use during intermediate-code generation.
- An important part of semantic analysis is type checking, where the compiler checks that each operator has matching operands.
Eg: The compiler must report an error if a floating-point number is used to index an array.
- The language specification may permit some type conversions like integer to float for float addition. This is known as coercions.
Eg: The operator * is applied to a floating-point number rate and an integer 60. The integer may be converted into a floating-point number by the operator **inttfloat** explicitly as shown in the figure below

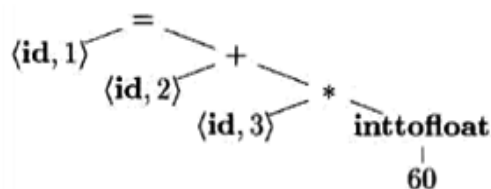


Fig: Semantic tree for position = initial + rate * 60

Intermediate Code Generation

- After syntax and semantic analysis of the source program, many compilers generate an explicit low-level or machine-like intermediate representation.
- Three-address code is one of the intermediate representations, which consists of a sequence of assembly-like instructions with three operands per instruction.
- The output of the intermediate code generator consists for position = initial + rate * 60 is given below:

```
t1 = inttofloat(60)
t2 = id3 * t1
t3 = id2 + t2
id1 = t3
```

Three address code instructions

- Each instruction has atmost one operator on right side
- These instructions fix the order in which operations are to be done. Eg: multiplication precedes addition in the source program.
- Some three address instructions have fewer than three operands.

Code Optimization

- Code optimization phase attempts to improve the intermediate code so that better target code will result.
- Better means faster, shorter etc.
- Optimization has to improve the efficiency of code so that the target program running time and consumption of memory can be reduced.
- Eg: inttofloat operation can be eliminated by replacing the integer 60 by the floating-point number 60.0. Moreover, t3 is used only once to transmit its value to id1 so the optimizer can transform it into the shorter sequence

```
t1= id3 * 60.0
```

```
id1 = id2 + t1
```

Code Generation

- The code generator takes as input an intermediate representation of the source program and maps it into the target language.
- The intermediate instructions are translated into sequences of machine instructions that perform the same task.
- For example, using registers R1 and R2, the intermediate code might get translated into the machine code

```
LDF R2, id3
MULF R2, R2, #60.0
LDF R1, id2
ADDF R1, R1, R2
STF id1, R1
```

- The first operand of each instruction specifies a destination.
- The F in each instruction tells us that it deals with floating-point numbers.
- LDF : loads contents of address id3 into register R2.
- The # signifies that 60.0 is to be treated as an immediate constant.
- STF : stores the value in register R1 into the address of id1

b. List and explain any three compiler construction tools.

(5)

Scanner generators that produce lexical analyzers from a regular-expression description of the tokens of a language. Unix has a tool for Scanner generator called **LEX**.

Parser generators that automatically produce syntax analyzers (parse tree) from a grammatical description of a programming language. Unix has a tool called **YACC** which is a parser generator.

Syntax-directed translation engines that produce collections of routines for walking a parse tree and generating intermediate code.

12. a What is a regular definition? Give the regular definition of an unsigned integer (7)

A regular definition is a way of formally defining a pattern using a combination of symbols and expressions. It's typically used in formal language theory to describe a set of strings that match the defined pattern.

The regular definition of an unsigned integer can be described as follows:

An unsigned integer is a sequence of one or more digits (0-9).

$$\begin{aligned}
 \text{digit} &\rightarrow 0|1|\dots|9 \\
 \text{digits} &\rightarrow \text{digit digit}^* \\
 \text{optionalFraction} &\rightarrow \cdot \text{digits} | \epsilon \\
 \text{optionalExponent} &\rightarrow (\epsilon (+|-|\epsilon) \text{digits}) | \epsilon \\
 \text{number} &\rightarrow \text{digits optionalFraction optionalExponent}
 \end{aligned}$$

ie, an optionalFraction is either a decimal point (dot) followed by one or more digits, or it is missing.

An optionalExponent, if not missing, is the letter E followed by an optional + or - sign, followed by one or more digits.

b. Express the role of transition diagrams in recognition of tokens.

(7)

The role of transition diagrams in token recognition can be summarized as follows:

1. **Token Definition**: Transition diagrams are used to define the structure and patterns of tokens in a formal and visual manner. Each token type (such as identifiers, keywords, numbers, operators, etc.) is associated with a specific transition diagram.
2. **State Transitions**: Transition diagrams represent the states of a finite-state machine, where each state corresponds to a particular configuration or context. The transitions between states are triggered by input symbols or characters.
3. **Token Recognition**: When the lexical analyzer reads characters from the input stream, it traverses the transition diagram based on the current state and the incoming input symbol. By following the transitions, the finite-state machine moves from one state to another, building the token incrementally.
4. **Accepting States**: Transition diagrams have accepting states or final states. When the finite-state machine reaches an accepting state, it indicates that a complete token has been recognized. At this point, the lexical analyzer can return the token to the parser for further processing.

5. **Error Handling:** Transition diagrams can incorporate error states or transitions to handle invalid or unrecognized input. If the lexical analyzer encounters a character that does not match any transition in the diagram, it can transition to an error state, indicating a lexical error.

13. a What is Recursive Descent parsing? List the challenges in designing such a parser? (4)

Recursive descent parsing is a top-down parsing technique commonly used in the implementation of parsers for programming languages. It involves constructing a parse tree from the top (starting with the input program) and recursively applying production rules to recognize the structure of the program.

Here's how Recursive Descent parsing works:

1. Start with the top-level non-terminal symbol of the grammar (usually the start symbol).
2. Write a parsing procedure for each non-terminal symbol that represents the production rules for that non-terminal.
3. Each parsing procedure corresponds to a specific production rule. It analyzes the input and recursively calls other parsing procedures for the non-terminals that appear in the production.
4. The parsing procedures continue recursively until the entire input is parsed, and a valid parse tree is constructed.

Challenges in designing a recursive descent parser include:

- ☐ Left Recursion: Left recursion in the grammar can lead to infinite recursion in the parsing procedure, causing the parser to enter an infinite loop. This issue must be addressed by transforming the grammar to eliminate left recursion
- ☐ Ambiguity: Ambiguous grammars have multiple possible parse trees for a single input string. Recursive Descent parsers cannot handle ambiguous grammars directly.
- ☐ Backtracking: Recursive Descent parsers typically employ a backtrack mechanism to try different parsing alternatives when encountering multiple choices in the grammar. Backtracking can be inefficient and may lead to exponential parsing time for certain input strings.

b. Consider the following grammar (10)

$E \rightarrow E \text{ or } T \mid T$

$T \rightarrow T \text{ and } F \mid F$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

- (i) Remove left recursion from the grammar.
- (ii) Construct a predictive parsing table.
- (iii) Justify the statement “The grammar is LL (1)”.

$E \rightarrow TE'$

$E' \rightarrow \text{or } TE' \mid \epsilon$

$T \rightarrow FT'$

$T' \rightarrow \text{and } FT' \mid \epsilon$

$F \rightarrow \text{not } F \mid (E) \mid \text{true} \mid \text{false}$

(ii) Constructing predictive parsing table:

	(	)	not	true	false	and	or	\$
E	$E \rightarrow TE'$		$E \rightarrow TE'$	$E \rightarrow TE'$	$E \rightarrow TE'$		$E \rightarrow TE'$	
E'		ϵ					$E' \rightarrow \text{or } TE'$	ϵ
T	$T \rightarrow FT'$		$T \rightarrow FT'$	$T \rightarrow FT'$	$T \rightarrow FT'$		$T \rightarrow FT'$	
T'		ϵ				$T' \rightarrow \text{and } FT'$	$T' \rightarrow \epsilon$	ϵ
F	$F \rightarrow (E)$		$F \rightarrow \text{not } F$	$F \rightarrow \text{true}$	$F \rightarrow \text{false}$			

The above table shows that the grammar is LL(1), meaning that it is possible to construct an LL(1) parser for this grammar. There are no conflicts in the table, so the grammar is unambiguous.

14. a What is Recursive Descent parsing? List the problems in designing such a parser. (4)

Recursive descent parsing is a top-down parsing technique commonly used in the implementation of parsers for programming languages. It involves constructing a parse tree from the top (starting with the input program) and recursively applying production rules to recognize the structure of the program.

Here's how Recursive Descent parsing works:

1. Start with the top-level non-terminal symbol of the grammar (usually the start symbol).
2. Write a parsing procedure for each non-terminal symbol that represents the production rules for that non-terminal.
3. Each parsing procedure corresponds to a specific production rule. It analyzes the input and recursively calls other parsing procedures for the non-terminals that appear in the production.

4. The parsing procedures continue recursively until the entire input is parsed, and a valid parse tree is constructed.

Problems in designing a recursive descent parser include:

- Left Recursion: Left recursion in the grammar can lead to infinite recursion in the parsing procedure, causing the parser to enter an infinite loop. This issue must be addressed by transforming the grammar to eliminate left recursion
- Ambiguity: Ambiguous grammars have multiple possible parse trees for a single input string. Recursive Descent parsers cannot handle ambiguous grammars directly.
- Backtracking: Recursive Descent parsers typically employ a backtrack mechanism to try different parsing alternatives when encountering multiple choices in the grammar. Backtracking can be inefficient and may lead to exponential parsing time for certain input strings.

b. Design a recursive descent parser for the grammar $S \rightarrow cAd$, $A \rightarrow ab \mid b$ (5)

o design a recursive descent parser for the given grammar, we need to create parsing functions for each non-terminal in the grammar: S and A. The parsing functions will recursively handle the production rules and check if the input string matches the grammar.

The grammar is as follows:

$S \rightarrow cAd$
 $A \rightarrow ab \mid b$

```
function S():
    if current_token == 'c':
        match('c')
        A()
        if current_token == 'd':
            match('d')
        else:
            report_error("Expected 'd'")
    else:
        report_error("Expected 'c'")
```

```
function A():
    if current_token == 'a':
        match('a')
    if current_token == 'b':
        match('b')
    else:
        report_error("Expected 'b'")
    elif current_token == 'b':
        match('b')
    else:
```

report_error("Expected 'a' or 'b'")

Find the FIRST and FOLLOW of the non-terminals S, A and B in the grammar

(5)

**$S \rightarrow aABe$
 $A \rightarrow Abc \mid b$
 $B \rightarrow d$**

$FIRST(S) = \{a\}$

$FIRST(A) = \{b\}$

$FIRST(B) = \{d\}$

$FOLLOW(S) = \{\$ \}$

$FOLLOW(A) = \{b, d\}$

$FOLLOW(B) = \{e\}$

**15. a. Construct the LR(0) set of items and their GOTO function for the grammar
(10)**

$S \rightarrow S S + \mid S S * \mid a$

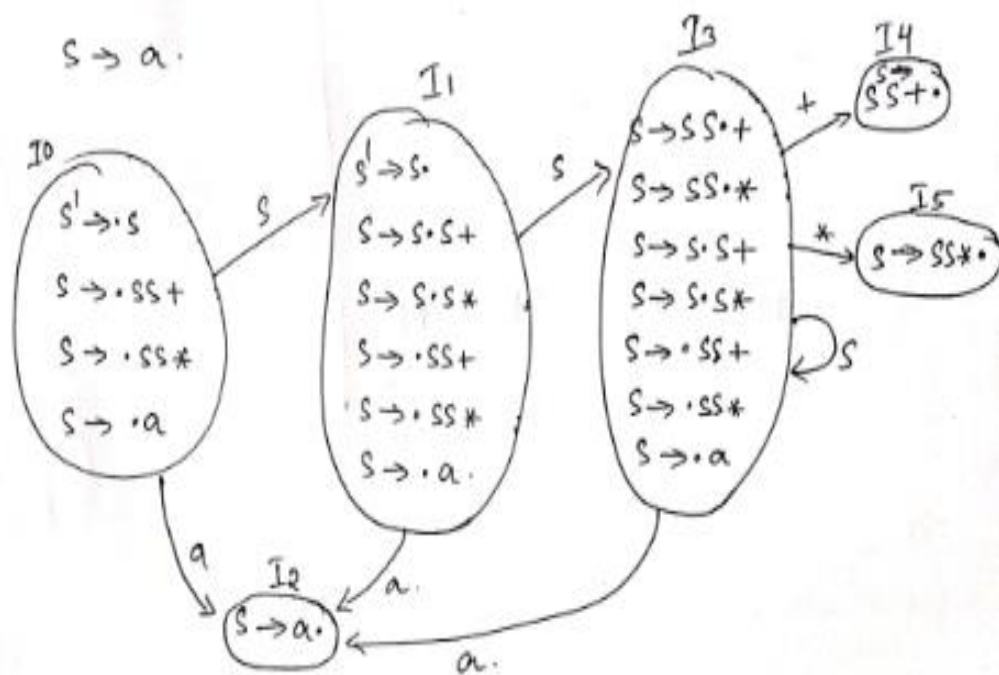
The augmented grammar is

$$S' \rightarrow S$$

$$S \rightarrow SS+$$

$$S \rightarrow SS*$$

$$S \rightarrow a.$$



I_0, I_1, I_2, I_3, I_4 and I_5 are the set of items in LR(0) collection.

$$I_1 = \text{Goto}(I_0, S)$$

$$I_2 = \text{Goto}(I_0, a) = \text{Goto}(I_1, a) = \text{Goto}(I_3, a)$$

$$I_3 = \text{Goto}(I_1, S) = \text{Goto}(I_3, S)$$

$$I_4 = \text{Goto}(I_3, +)$$

$$I_5 = \text{Goto}(I_3, *)$$

b. Is the grammar SLR? Justify your answer

(4)

Follow(S) = {+, *, a, \$}

SLR Parsing table.

States	a	Action +	*	\$	Go to S
0	S2				1
1	S2			accept.	3
2	r3	r3	r3	r3	
3	S2	S4	S5		3
4	r1	r1	r1	r1	
5	r2	r2	r2	r2	

① $S \rightarrow SS +$

② $S \rightarrow SS *$

③ $S \rightarrow a$

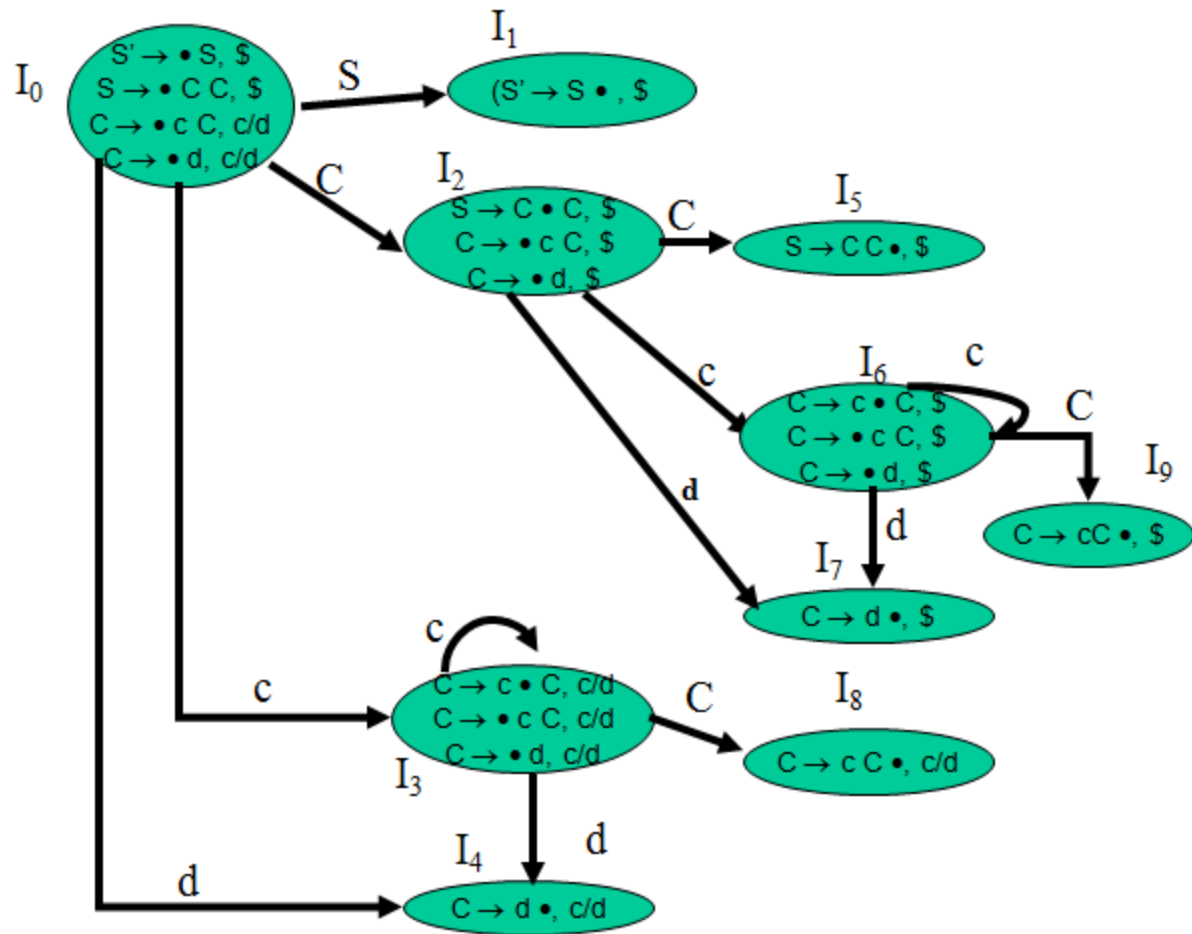
Since there is no multiple entries in any cell of the above table, the given grammar is SLR.

16.a. Identify LR(1) items for the grammar

(7)

$S \rightarrow CC$

$C \rightarrow cC \mid d$

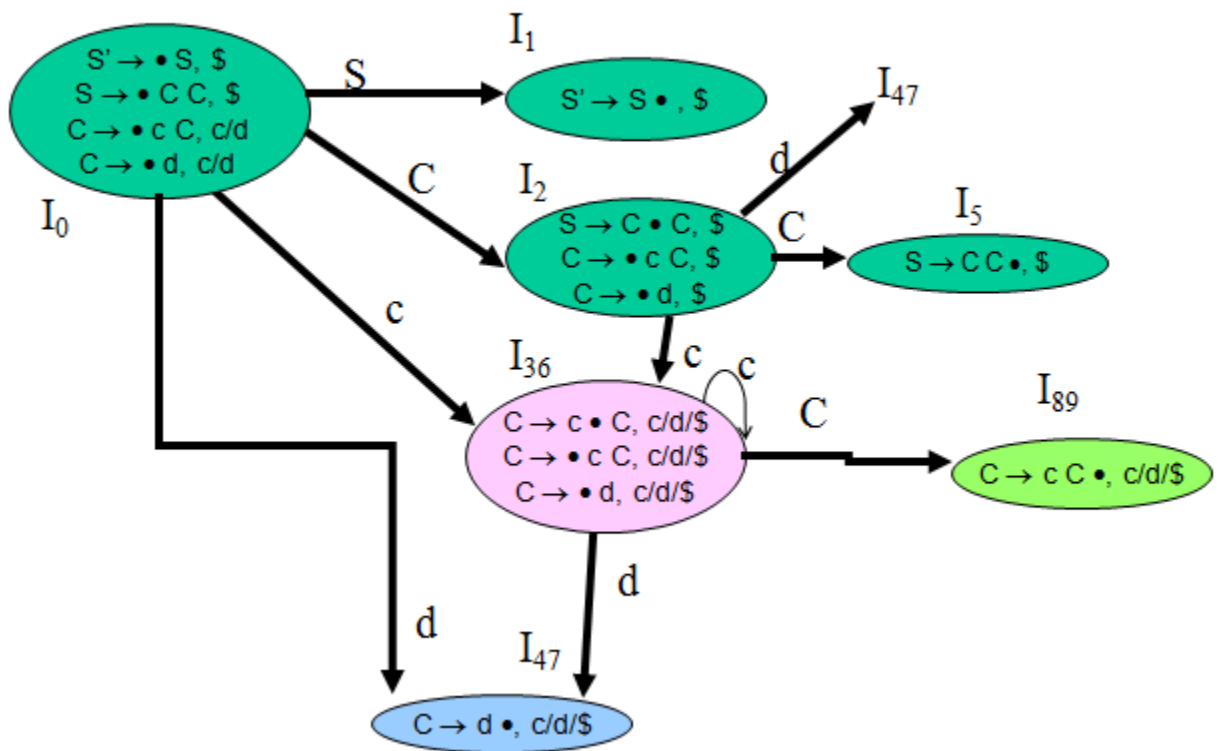


$I_0, I_1, I_2, I_3, I_4, I_5, I_6, I_7, I_8, I_9$ are the set of LR(1) items.

b. Construct LALR table for the above grammar

(7)

LALR Automaton



LALR Table

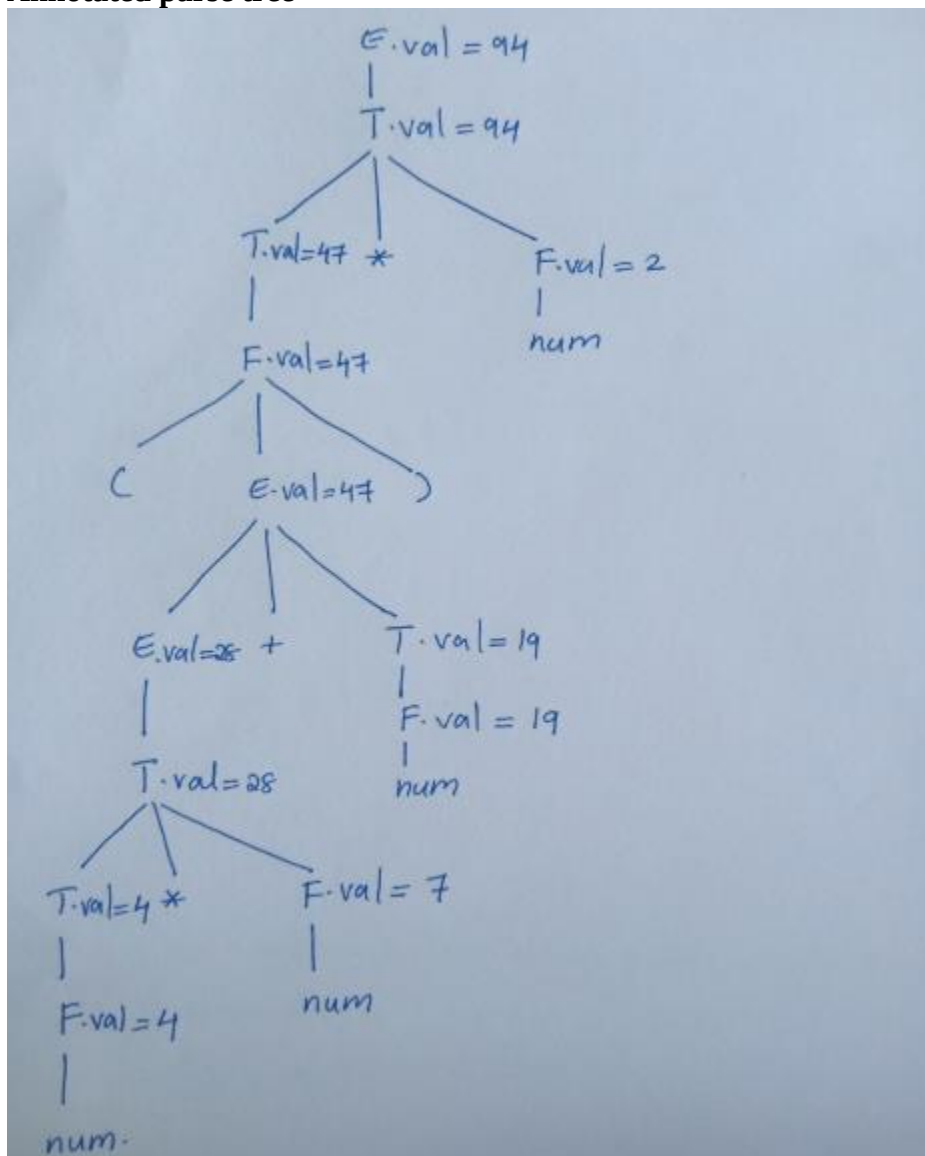
	c	d	\$	S	C
0	s36	s47		1	2
1			acc		
2	s36	s47			5
36	s36	s47			89
47	r3	r3	r3		
5			r1		
89	r2	r2	r2		

17 a. Design a Syntax Directed Translator(SDT) for the arithmetic expression $(4 * 7 + 19) * 2$ and draw an annotated parse tree for the same. (8)

let's define the grammar rules for the arithmetic expressions:

$E \rightarrow E_1 + T \{E.val = E_1.val + T.val\}$
 $E \rightarrow T \{E.val = T.val\}$
 $T \rightarrow T_1 * F \{T.val = T_1.val * F.val\}$
 $T \rightarrow F \{T.val = F.val\}$
 $F \rightarrow (E) \{F.val = E.val\}$
 $F \rightarrow \text{num} \{F.val = \text{digit.lexval}\}$

Annotated parse tree



b. Consider the grammar with following translation rules and E as the start symbol (6)

$E \rightarrow E1 \# T \{ E.value = E1.value \times T.value ; \}$

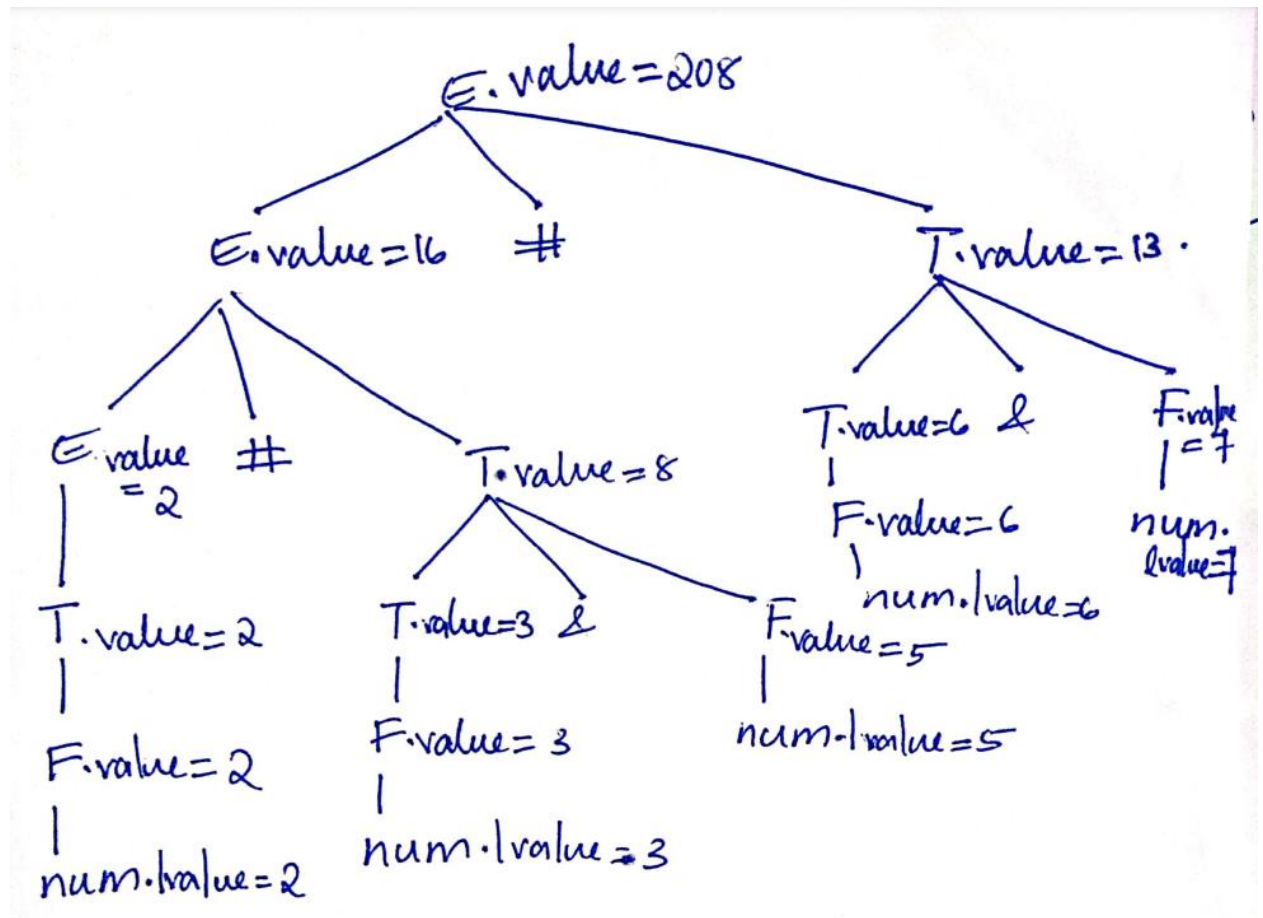
$\quad \quad \quad | T \{ E.value = T.value ; \}$

$T \rightarrow T1 \& F \{ T.value = T1.value + F.value ; \}$

$\quad \quad \quad | F \{ T.value = F.value ; \}$

$F \rightarrow \text{num} \{ F.value = \text{num}.lvalue ; \}$

Compute E.value for the root of the parse tree for the expression $2 \# 3 \& 5 \# 6 \& 7$



18 a. Write Syntax Directed Translator (SDT) and parse tree for infix to postfix translation of an expression. (8)

A simple translation scheme that converts infix expressions to the corresponding postfix expressions

$E \rightarrow E + T \{ \text{print} (" + ") \}$

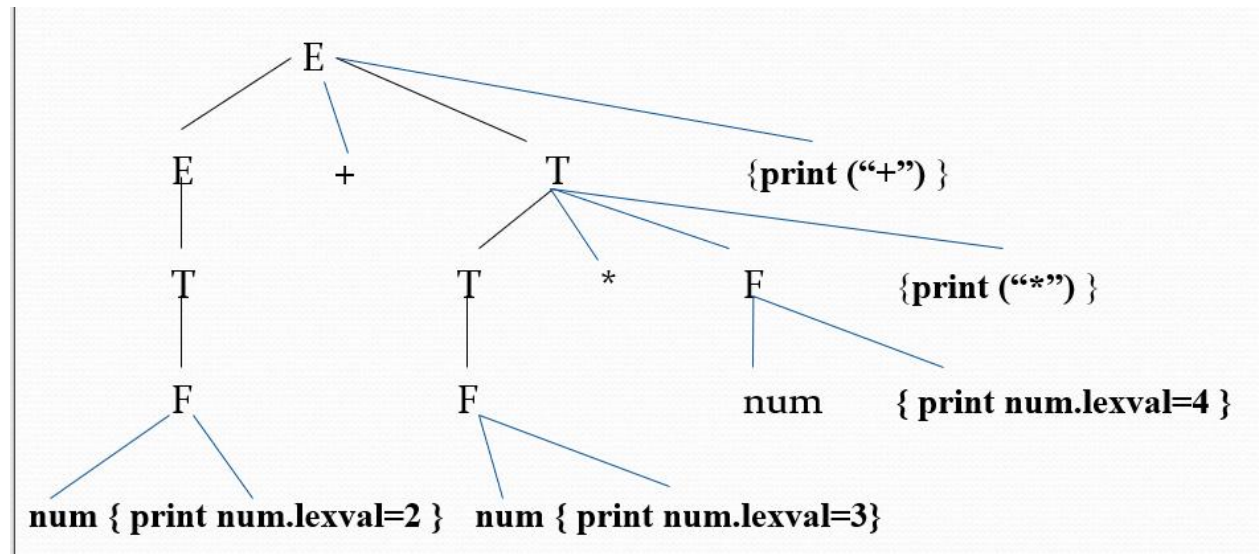
$E \rightarrow T$

$T \rightarrow T * F \{ \text{print} (" * ") \}$

$T \rightarrow F$

F->num { print num.lexval }

Eg: Conversion of 2+3*4 into postfix notation using above SDT



The depth first traversal of the parse tree (executing the semantic actions in that order) will produce the postfix representation of the infix expression which is 234*+

b. Explain the storage allocation strategies.

(6)

The different Storage Allocation Strategies are:

1. Static Allocation:

- Names are bound to storage at compile time.
- Since bindings do not change at run time, every time a procedure is activated, its names are bound to the same storage location.
- This property allows values of local names to be retained across activations of a procedure.
- The compiler uses the type of a name (retrieved from the symbol table) to determine storage size required.
- The required number of bytes is set aside for the name.
- The address of the storage is fixed at compile time.

2. Stack Allocation

- Dynamic allocation based on the idea of a control stack. Storage is organized as a stack.
- Activation records are pushed to and popped from stack as activations begin and end.
- Storage for the locals in each call of a procedure is contained in the activation

record for that call.

- This means locals are bound to fresh storage on every call, because a new AR is pushed onto the stack when a call is made.
- The values of the locals are deleted when the activation ends, because AR is popped.
- **top** marks the top of the stack.
- To allocate a new activation record, just increment **top** by the size of record.
- To deallocate an existing activation record, just decrement **top** by the size of record.
- Eg: if procedure q has an activation record of size a, then top is incremented by 'a' just before the target code of q is executed. And when control returns from q, top is decremented by 'a'.

3. Heap Allocation

- Allocates pieces of contiguous storage, as needed for AR or other objects.
- Pieces may be deallocated in any order.
- So over time heap will consist of alternate areas that are free and in use. It is left to the heap manager to make use of this space.

19. a. Describe the principal sources of optimization

(7)

1. Function preserving transformations

2. Loop optimization

a. Common subexpression elimination

a. Code motion

b. Copy propagation

b. Induction variable elimination

c. Dead code elimination

c. Reduction in strength

d. Constant folding

Function-Preserving Transformations

- There are a number of ways in which a compiler can improve a program without changing the function it computes.
- The common function preserving transformations are:
 1. Common subexpression elimination
 2. Copy propagation
 3. Dead-code elimination
 4. Constant folding

1. Common Subexpression Elimination

- An occurrence of an expression E is called a common sub-expression if E was previously computed, and the values of variables in E have not changed since the previous computation.
- We can avoid recomputing the expression if we can use the previously computed value.
- For example

```
t1: = 4*i
t2: = a [t1]
t3: = 4*j
t4:= 4*i
t5: = n
t6: = b [t4] +t5
```
- The above code can be optimized using the common sub-expression elimination as `t1: = 4*i   t2: = a [t1]   t3: = 4*j   t5: = n   t6: = b [t1] +t5`
- The common sub expression `t4:=4*i` is eliminated as its computation is already in `t1`. And value of `i` is not been changed from definition to use.

2. Copy Propagation

- Assignments of the form `f := g` are called copy statements, or copies for short.
- The idea behind the copy-propagation transformation is to use `g` for `f`, whenever possible after the copy statement `f:= g`.
- Copy propagation means use of one variable instead of another.
- This may not appear to be an improvement, but it gives us an opportunity to eliminate `x`.
- For example:

```
x=Pi;
.....
A=x*r*r;
```
- The optimization using copy propagation can be done as follows:

```
A=Pi*r*r;      Here the variable x is eliminated
```

3. Dead-Code Eliminations:

- A variable is live at a point in a program if its value can be used subsequently; otherwise, it is dead at that point.

- A related idea is dead or useless code, statements that compute values that never get used.
- While the programmer is unlikely to introduce any dead code intentionally, it may appear as the result of previous transformations.
- An optimization can be done by eliminating dead code.
- Example:

```
i=0;
if(i==1) { a=b+5; }
```

- Here, 'if' statement is dead code because this condition will never get satisfied.

4. Constant folding:

- If all operands are constants in an expression, then it can be evaluated at compile time itself. The result of the operation can replace the original evaluation in the program.
- This will improve the run time performance by avoiding evaluation at compile- time.
- For example,
 $a=3.14157/2$ can be replaced by $a=1.570$ there by eliminating a division operation.

Loop Optimizations:

- The running time of a program may be improved if we decrease the number of instructions in an inner loop, even if we increase the amount of code outside that loop.
- Three techniques are important for loop optimization:
 - A) Code Motion, which moves code outside a loop;
 - B) Induction-variable elimination, which we apply to replace variables from inner loop.
 - C) Reduction in Strength, which replaces expensive operation by a cheaper one, such as a multiplication by an addition.

A) Code Motion (or Frequency Reduction)

- An important modification that decreases the amount of code in a loop is code motion.
- A fragment of code that resides in the loop and computes the same value at each iteration is called a loop-invariant code. This code can be moved out of the loop by saving it to be computed only once, rather than with each iteration.

- For example, evaluation of limit-2 is a loop-invariant computation in the following while-statement:

`while (i <= limit-2) /* statement does not change limit*/`

Code motion will result in the following

`t= limit-2;`

`while (i<=t) /* statement does not change limit or t */`

B) Induction Variables :

- A variable is called an induction variable if its value is altered within the loop by a loop-invariant value.

Example:

The code fragment below has three induction variables (i1, i2, and i3) that can be replaced with one induction variable, thus eliminating two induction variables.

```
int a[SIZE];
int b[SIZE];

void f (void)
{
    int i1, i2, i3;

    for (i1 = 0, i2 = 0, i3 = 0; i1 < SIZE; i1++)
        a[i2++] = b[i3++];
    return;
}
```

The code fragment below shows the loop after induction variable elimination.

```
int a[SIZE];
int b[SIZE];

void f (void)
{
    int i1;

    for (i1 = 0; i1 < SIZE; i1++)
        a[i1] = b[i1];
    return;
}
```

Notes:

Induction variable elimination can reduce the number of additions (or subtractions) in a loop, and improve both run-time performance and code space.

- Reduction in strength replaces expensive operations by equivalent cheaper ones on the target machine.
- Certain machine instructions are considerably cheaper than others and can often be used as special case of more expensive operators.
 - For example, x^2 is invariably cheaper to implement as $x*x$ than as a call to an exponentiation routine.

- Fixed-point multiplication or division by a power of two is cheaper to implement as a shift.
- Floating-point division by a constant can be implemented as multiplication by a constant, which may be cheaper.

Example: Replacement of a multiplication by an addition

- Here, high strength operator '*' in 'i*5' occurring inside loop is replaced by a low strength operator '+' in 'itemp+5'

```

for i := 1 to 10 do
begin
  - - -
  k := i*5;
  - - -
end;

```

⇒

```

itemp := 5;
for i := 1 to 10 do
begin
  - - -
  k := itemp;
  - - -
  itemp := itemp+5;
end;

```

b. Illustrate the optimization of basic blocks with examples.

(7)

There are two types of basic block optimizations or transformations.

They are:

1. Structure-Preserving Transformations
2. Algebraic Transformations

1. Structure-Preserving Transformations:

- The primary Structure-Preserving Transformation on basic blocks are:
 - Common sub-expression elimination.
 - Dead code elimination.
 - Renaming of temporary variables
 - Interchange of two independent adjacent statements.

➤ Common sub-expression elimination:

Common sub expressions need not be computed over and over again. Instead they can be computed once and kept in store from where it's referenced when encountered again

B_5

```
t6 := 4*i  
x := a[t6]  
t7 := 4*i  
t8 := 4*j  
t9 := a[t8]  
a[t7] := t9  
t10 := 4*j  
a[t10] := x  
goto B2
```

(a) Before

B_5

```
t6 := 4*i  
x := a[t6]  
t8 := 4*j  
t9 := a[t8]  
a[t6] := t9  
a[t8] := x  
goto B2
```

(b) After

Fig. 10.6. Local common subexpression elimination.

➤ **Dead code elimination:**

It's possible that a large amount of dead (useless) code may exist in the program. This might be especially caused when introducing variables and procedures as part of construction or error-correction of a program – once declared and defined, one forgets to remove them in case they serve no purpose. Eliminating these will definitely optimize the code.

➤ **Renaming of temporary variables:**

- A statement $t := b + c$ where t is a temporary name can be changed to $u := b + c$ where u is another temporary name, and change all uses of t to u .
- In this we can transform a basic block to its equivalent block called normal-form block.

➤ **Interchange of two independent adjacent statements:**

- Two statements

$t_1 := b + c$

$t_2 := x + y$

can be interchanged or reordered in its computation in the basic block when value of t_1 does not affect the value of t_2 .

2. Algebraic Transformations

- Algebraic identities represent another important class of optimizations on basic blocks.

— Example:

$$x + 0 = 0 + x = x$$

$$x - 0 = x$$

$$x * 1 = 1 * x = x$$

$$x / 1 = x$$

- Another class of algebraic optimisation includes replacing an expensive operation by cheaper ones i.e. reduction in strength.

— Example:

$$x ** 2 = x * x$$

$$2.0 * x = x + x$$

$$x / 2 = x * 0.5$$

- Another class of algebraic related optimizations is constant folding.
 - Here we evaluate constant expressions at compile time and replace the constant expressions by their values.
 - Thus the expression $2 * 3.14$ would be replaced by 6.28.

20. a Write the code generation algorithm and explain the getreg function (6)

Code Generation Algorithm

- The algorithm takes as input a sequence of three-address statements constituting a basic block.
- For each three-address statement of the form $x := y \text{ op } z$, perform the following actions:
 1. Invoke a function **getreg** to determine the location L where the result of the computation $y \text{ op } z$ should be stored.
 2. Consult the address descriptor for y to determine y' , the current location of y . Prefer the register for y' if the value of y is currently both in memory and a register. If the value of y is not already in L , generate the instruction $\text{MOV } y', L$ to place a copy of y in L .
 3. Generate the instruction $\text{OP } z', L$ where z' is a current location of z . Prefer a register to a memory location if z is in both. Update the address descriptor of x to indicate that x is in location L . If x is in L , update its descriptor and remove x from all other descriptors.
 4. If the current values of y or z have no next uses, are not live on exit from the block, and are in registers, alter the register descriptor to indicate that, after execution of $x := y \text{ op } z$, those registers will no longer contain y or z .

getreg function

Code generator uses *getReg* function to determine the status of available registers and the location of name values. *getReg* works as follows:

- If variable Y is already in register R, it uses that register.
- Else if some register R is available, it uses that register.
- Else if both the above options are not possible, it chooses a register that requires minimal number of load and store instructions. That is, it takes a register which is already occupied, move its contents into some memory using the instruction MOV R, M. and then uses the register R.

b. Generate the target code for the following statement

(8)

$d := (a-b) + (a-c) + (a-c)$

- The assignment $d := (a-b) + (a-c) + (a-c)$ might be translated into the following three-address code sequence:

$t := a - b$
 $u := a - c$
 $v := t + u$
 $d := v + u$

with d live at the end

Code sequence for the example is:

Statements	Code Generated	Register descriptor	Address descriptor
		Register empty	
$t := a - b$	MOV a, R ₀ SUB b, R ₀	R ₀ contains t	t in R ₀
$u := a - c$	MOV a, R ₁ SUB c, R ₁	R ₀ contains t R ₁ contains u	t in R ₀ u in R ₁
$v := t + u$	ADD R ₁ , R ₀	R ₀ contains v R ₁ contains u	u in R ₁ v in R ₀
$d := v + u$	ADD R ₁ , R ₀ MOV R ₀ , d	R ₀ contains d	d in R ₀ d in R ₀ and memory